Problem Solving Task

CaB420 assessment 1a - Problem Solving Task

Oliver Stewart (n11588608)

april 3rd, 2026

Contents

1 Purpose

2 Problem Summaries

2.1 Problem 1. Socio-economic affect on Crime Regression

2.1.1 Data Description

2.1.2 Task Description

2.2 Problem 2. Land Type Classification

2.2.1 Data Description

2.2.2 Task Description

2.3 Problem 3. Training and adapting Image Classification Deep Networks

2.3.1 Data Description

2.3.2 Task Description

3 Problem 1. Socio-economic affect on Crime Regression

3.1 Data Pre-processing

3.1.1 Standardization

3.2 Methodology

3.2.1 Linear

3.2.2 Ridge

3.2.3 Lasso

3.3 Results and Evaluation

3.3.1 Evaluation

3.4 Ethical Considerations

4 Problem 2. Land Type Classification

4.1 Data Pre-processing

4.2 Methodology

4.2.1 Hyper-parameter Selection

4.2.2 Final Model Parameters

4.2.3 Hyper-parameter Discussion

*4.2.4 **K-Nearest Neighbors (KNN)***

*4.2.5 **Support Vector Machine (SVM)***

*4.2.6 **Random Forest (RF)***

4.3 Results and Evaluation

5 Problem 3. Training and adapting Image Classification Deep Networks

5.1 Data Pre-processing

5.2 Methodology

5.2.1 Network Design

5.2.2 Training Approach

5.2.3 Data Augmentation

5.3 Results and Evaluation

6 Appendicies

6.1 Appendix A - Iterative Log-space Search for Ridge and Lasso Regression in Problem 1.

6.1.1 Ridge

6.1.2 Lasso

6.2 Appendix B - Histograms before and after PowerTransformer in Problem 2.

1 Purpose

assessment 1a is a problem-solving task that asks us to individually answer three questions where we apply various machine learning methods to different datasets, and analyse and interpret results. This document presents the responses to these questions in a concise report.

Brief quotes for the task descriptions are below:

2 Problem Summaries

2.1 Problem 1. Socio-economic affect on Crime Regression

The purpose is to explore the link between the various socio-economic factors and crime.

2.1.1 Data Description

The data in the Q1 directory in CaB420 assessment 1a Data.zip contains socio-economic data from the 1990 US census for various US communities, and the number of violent crimes per capita (in the column ViolentCrimesPerPop). The provided data has been split into training (communities train), validation (communities val), and testing (communities test) sets.

2.1.2 Task Description

Using this data you are to:

- Train a linear regression model to predict the number of violent crimes per captia from the socio-economic data.
- Train a Ridge regression model to predict the number of violent crimes per captia from the socio-economic data.
- Train a Lasso regression model to predict the number of violent crimes per captia from the socio-economic data.

2.2 Problem 2. Land Type Classification

Land use classification is an important task to understand our changing environment. One approach to this involves the use of data from aerial sensors that captures different spectral reflectance properties of the ground below. From this data, the land type can be classified.

2.2.1 Data Description

You have been provided with training, validation and testing data (Q2/training.csv, Q2/validation.csv and Q2/testing.csv) that include 27 spectral properties and an overall classification of land type, which can be one of:

- s : ‘Sugi’ forest;
- h : ‘Hinoki’ forest;
- d : ‘Mixed deciduous’ forest;
- o : ‘Other’ non-forest land

2.2.2 Task Description

Using the provided data as-is, you are to train three multi-class classifiers to classify land type from the spectral data. These classifiers are to be:

- a K-Nearest Neighbours Classifier;
- a Random Forest; and
- an ensemble of Support Vector Machines.

2.3 Problem 3. Training and adapting Image Classification Deep Networks

When training deep neural networks, the availability of data is a frequent challenge. as such, methods including fine tuning and data augmentation are common practices to address data challenges.

2.3.1 Data Description

You have been provided with three portions of data from the Street View House Numbers (SVHN) dataset, a 'real world' MNIST style dataset. The three data portions are:

- a training set, Q3/q3 train.mat, containing 1, 000 samples distributed across the 10 classes.
- a validation set, Q3/q3 validation.mat, containing 1, 000 samples distributed across the 10 classes.
- a testing set, Q3/q3 test.mat, 10, 000 samples distributed across the 10 classes.

Data sets do no overlap, and have been extracted randomly from the original SVHN test set. Note that the training set being significantly smaller than the test set is by design, and is not an error. The provided template code implements an SVM to classify this data, and provides code to time how long the SVM takes during training and testing

2.3.2 Task Description

Using these datasets and the provided code, you are to:

- Design/select a DCNN architecture, and using this network:
- Train a model from scratch, using no data augmentation, on the provided abridged SVHN training set.
- Train a model from scratch, using the data augmentation of your choice, on the provided abridged SVHN training set.
- Compare the performance of two DCNNs (with and without augmentation), considering the accuracy, training time and inference time (i.e. time taken to evaluate the models), using the provided test set.

all models should be evaluated on the provided SVHN test set, and their performance should be compared. DCNN architectures may be taken from lecture examples or practical solutions. Your selection of model may take computational constraints into consideration.

3 Problem 1. Socio-economic affect on Crime Regression

3.1 Data Pre-processing

3.1.1 Standardization

In this problem, sklearn's PowerTransform was selected to transform the original data because, as informed by exploratory data analysis, the data contains vastly different distributions, e.g. some are log-normal, some are just skewed normal and a regular Z-score normalisation technique would under-perform on this dataset. The only alternative to properly normalise and standardise this dataset would be a manual trial and error approach on every individual variable in the dataset. The results of this method can be seen in Figure 1.1.

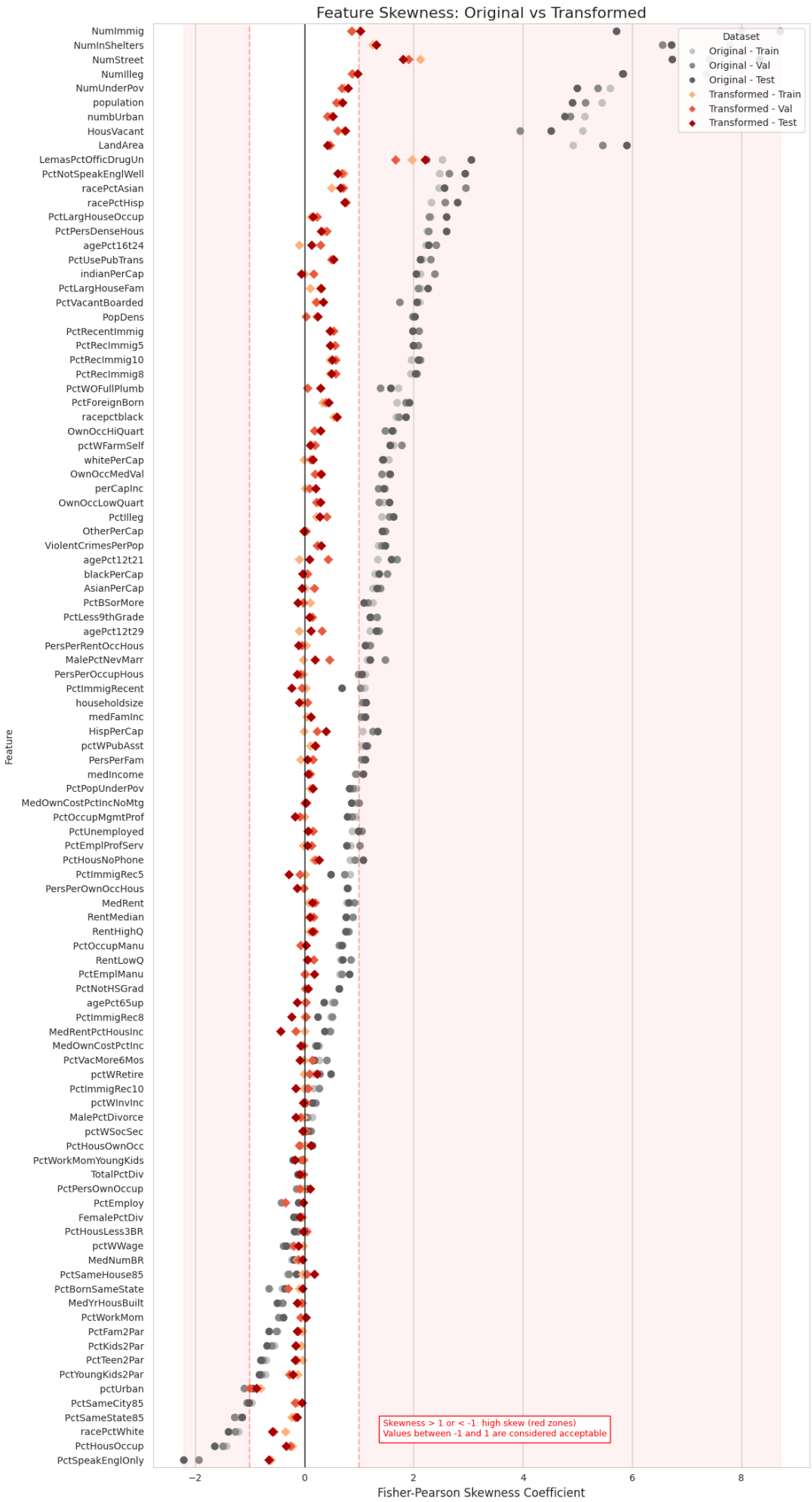


Figure 1.1 Fisher-Pearson Feature Skewness: Original vs. Transformed

3.2 Methodology

3.2.1 Linear

a standard OLS model was fit using `statsmodels`, using all 100 predictor variables on the PowerTransformed data. The model was evaluated using RMSE, R^2, and MaE across the train, validation, and test sets, with residual distribution and Q-Q plots used to verify model assumptions.

3.2.2 Ridge

Ridge regression (L2 regularisation) was fit using `statsmodels` with `L1_wt=0`. alpha was selected via an iterative log-space search over 10 passes, each refining the search range around the best validation RMSE from the previous pass (see Appendix A).

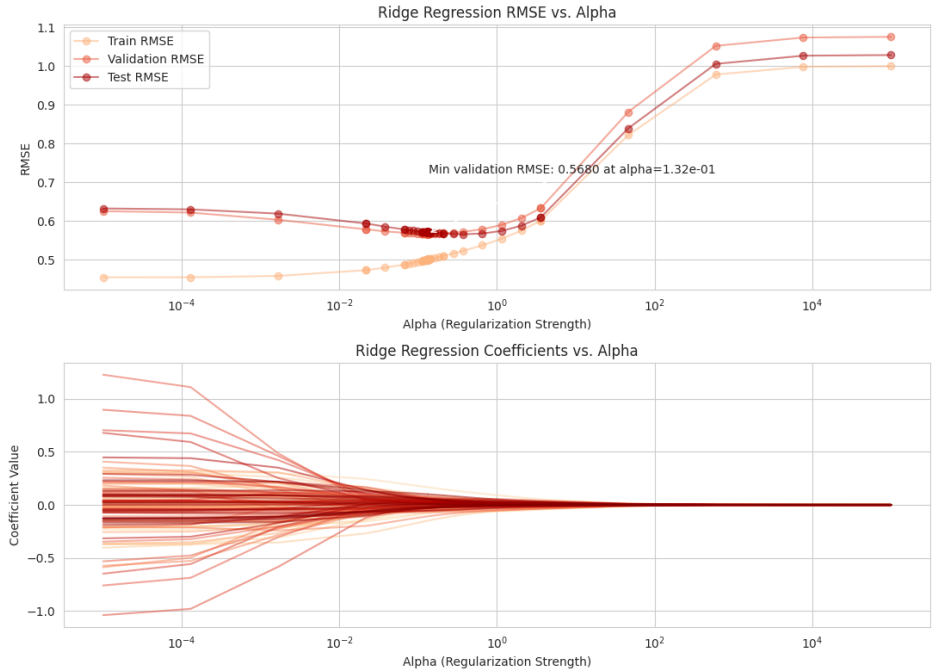


Figure 1.2 Ridge Regression Iterative Log-Space Search Results

Figure 1.2 shows the output of this iterative log-space search, alpha converged to ~0.1323, achieving a validation RMSE of 0.5680. all 100 parameters were retained but with shrunk coefficients.

3.2.3 Lasso

Lasso regression (L1 regularisation) was fit using `statsmodels` with `L1_wt=1`, using the same iterative alpha search strategy as Ridge (see Appendix A).

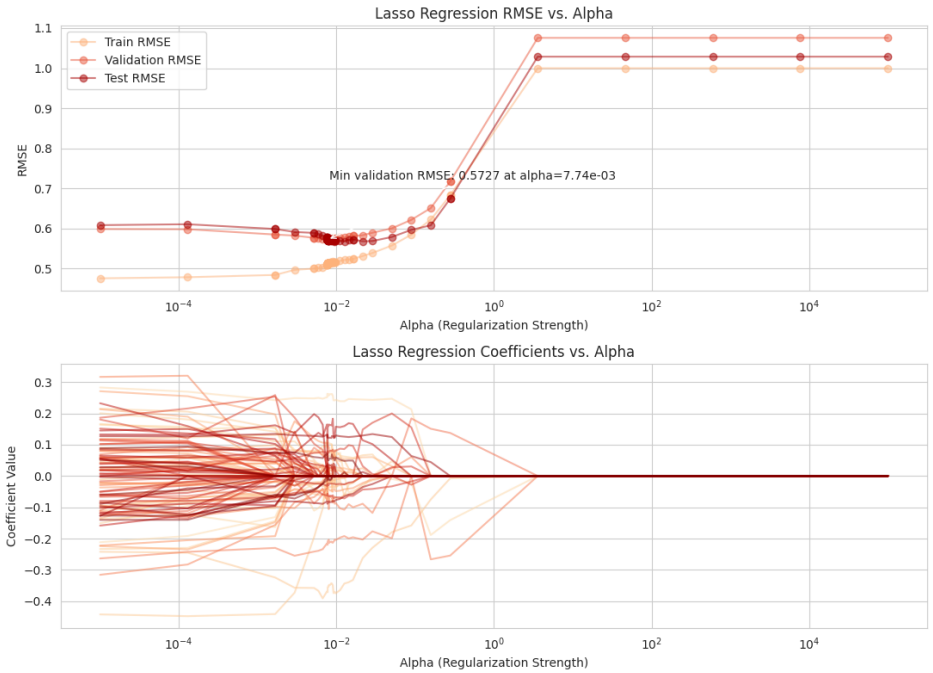


Figure 1.3 Lasso Regression Iterative Log-Space Search Results

Figure 1.3 shows the output of this iterative log-space searcd, alpha converged to ~0.00774, achieving a validation RMSE of 0.5727. 65 of the 100 coefficients were zeroed out, leaving 35 active predictors and providing implicit feature selection. The more aggressive coefficient zeroing is also observed in the Coefficients vs Alpha plots.

3.3 Results and Evaluation

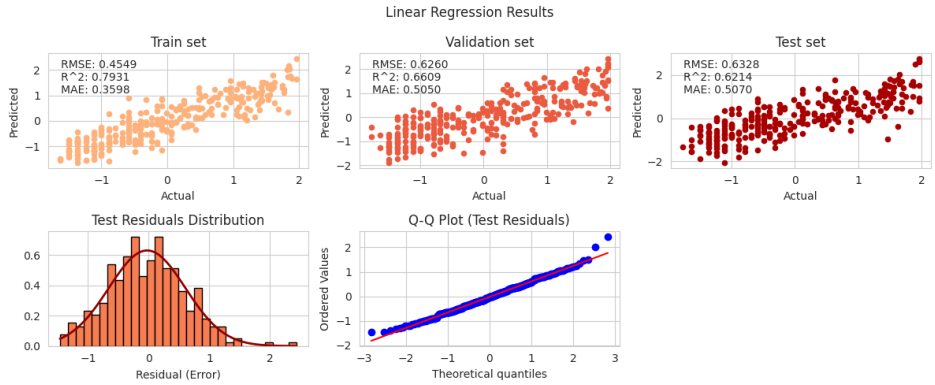


Figure 1.4 Linear Regression Model Results

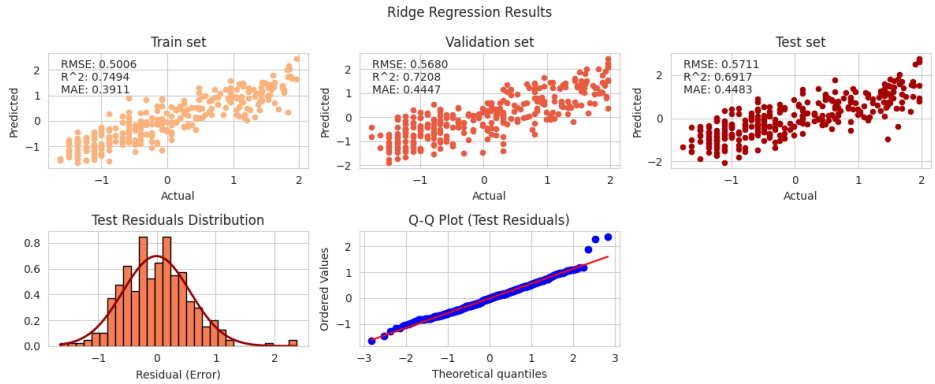


Figure 1.5 Ridge Regression Model Results

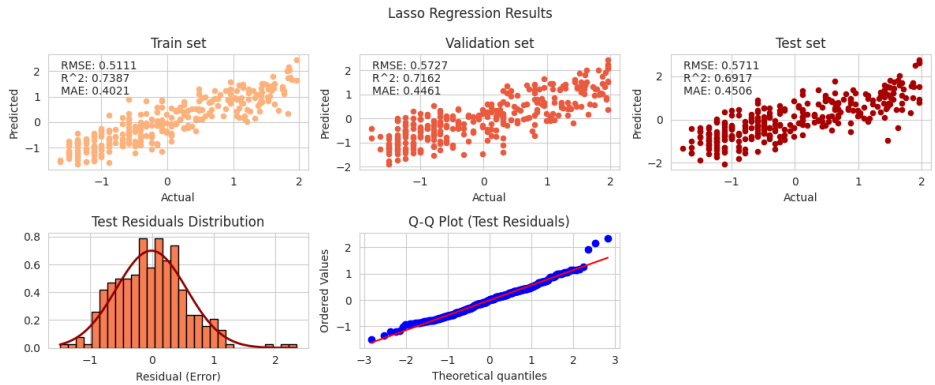


Figure 1.6 Lasso Regression Model Results

3.3.1 Evaluation

From the above plots, Figures 1.4 to 1.6, it can be seen that all three models generalise well across train, validation, and test sets, with normally distributed residuals confirming model assumptions. Linear regression performed the worst, while Ridge and Lasso achieved near-identical predictive accuracy. Lasso is the preferred model - it matches Ridge's performance with only 35 active predictors versus 100, yielding a significantly less complex model.

Model	RMSE	R^2	MaE	Params
Linear	0.632848	0.621414	0.506974	100
Ridge	0.571134	0.691652	0.448261	100
Lasso	0.571111	0.691677	0.450613	35

3.4 Ethical Considerations

Using socio-economic indicators to predict crime rates carries significant ethical risk. Many variables in this dataset are demographic proxies (e.g. race, income, housing) that reflect systemic inequalities rather than individual behaviour. Predictions derived from such data can reinforce and amplify existing biases - for example, by directing policing resources toward already over-policed communities. additionally, the data originates from the 1990 US census, which limits its relevance to present-day communities. This model should be treated as an analytical exercise only; deploying it to inform policy or resource allocation decisions would be inappropriate without extensive bias auditing and community consultation.

4 Problem 2. Land Type Classification

4.1 Data Pre-processing

The 27 features show non-normal distributions (a mix of log-normal, right-skewed, and near-normal shapes), which was confirmed via EDA

histograms across all three splits (see Appendix B). A standard Z-score normalisation would under-correct these skews, so sklearn's PowerTransformer (Yeo-Johnson method) was applied. The transformer was fitted exclusively on the training set and then applied to the validation and test sets to avoid data leakage (see Figure 2.1 to see the before and after using PowerTransformer). No samples were removed as the dataset contained no null values or infinite values, and the four target classes were reasonably balanced meaning no data augmentation was required.

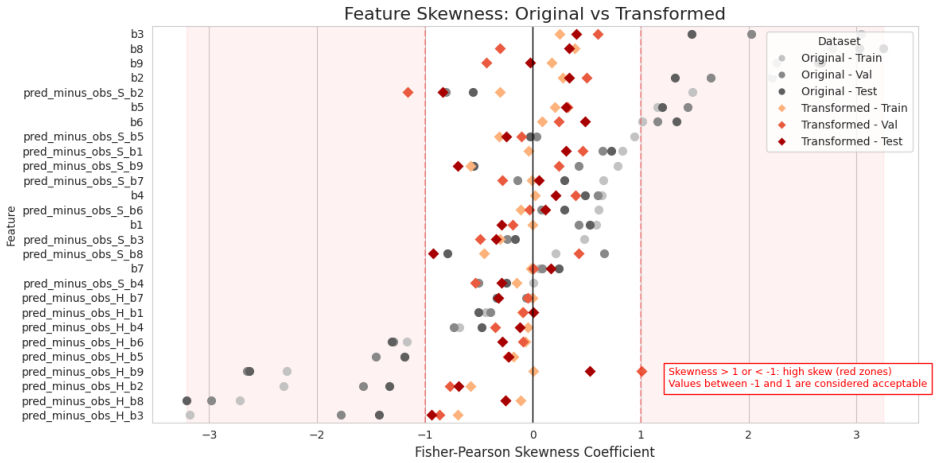


Figure 2.1 Feature distributions before and after PowerTransformer normalisation.

4.2 Methodology

4.2.1 Hyper-parameter Selection

Hyper-parameters for all three classifiers were selected via cross-validated search on the training set, with final model selection guided by validation accuracy.

-*K-Nearest Neighbours (KNN):** A RandomizedSearchCV (20 iterations, 5-fold CV) was run over

$$\text{n_neighbors} \in [1, 20]$$

and

$$\text{weights} \in \{uniform, distance\}$$

Randomised search was chosen over exhaustive grid search because the parameter space is small enough that 20 samples gives broad coverage without the computational overhead of evaluating every combination.

-*Support Vector Machine (SVM):** A GridSearchCV (5-fold CV) was run across three kernel types simultaneously. The hyperparameter space S is defined as the union of three disjoint sets based on the kernel type:

$$S = S_{linear} \cup S_{rbf} \cup S_{poly}$$

Where the parameter grids are defined as:

-*Linear Kernel:**

$$S_{linear} = \{(C, \text{kernel}) \mid C \in \{0.1, 1, 10, 100, 1000\}, \text{kernel} = \text{'linear'}\}$$

-*RBF Kernel:**

$$S_{rbf} = \{(C, \gamma, \text{kernel}) \mid C \in \{0.1, \ldots, 1000\}, \gamma \in \{0.1, 0.01, 0.001, 0.0001\}, \text{kernel} =$$

-*Polynomial Kernel:**

$$S_{poly} = \{(C, d, \text{kernel}) \mid C \in \{0.1, \ldots, 1000\}, d \in \{3, 4, 5, 6\}, \text{kernel} = \text{'poly'}\}$$

Full grid search was used here because the multi-kernel structure made it important to evaluate all kernel-parameter combinations exhaustively. sklearn's multiclass SVC uses a one-vs-one (OvO) strategy internally, fitting C(4,2) = 6 binary classifiers that vote to produce the final prediction - effectively an ensemble of SVMs.

-*Random Forest (RF):** A HalvingGridSearchCV (5-fold CV) was run over the hyperparameter space S, defined as the Cartesian product of the sets for each parameter:

$$S = \{(d, m, n) \mid d \in \text{max_depth}, m \in \text{min_samples_split}, n \in \text{n_estimators}\}$$

Where:

- $\text{max_depth} = \{2, 4, \text{None}\}$
- $\text{min_samples_split} = \{5, 10\}$
- $\text{n_estimators} = \{25, 50, 100\}$

Successive halving was chosen to efficiently prune poorly performing configurations early, reducing total training time relative to full grid search.

Figure 2.2 shows the validation accuracy surface for the SVM's C and γ search on the RBF kernel, illustrating how performance peaks at moderate values before degrading.

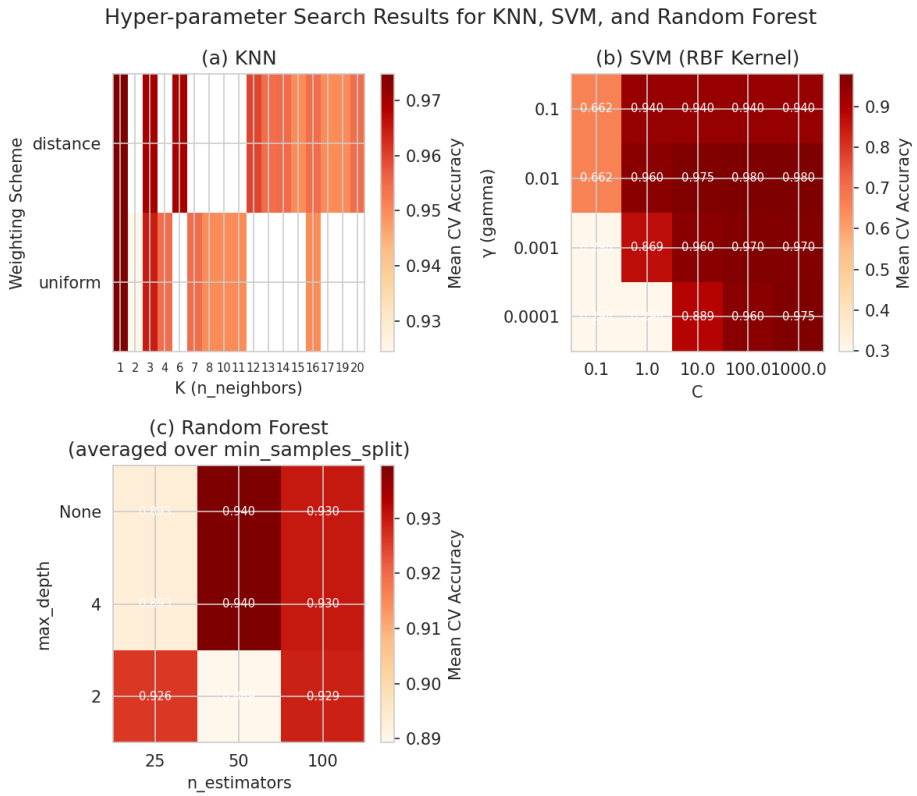


Figure 2.2 Hyper-parameter search results for KNN, SVM, and Random Forest.

4.2.2 Final Model Parameters

Model	Parameter	Value
KNN	`n_neighbors`	1
KNN	`weights`	distance
SVM	`kernel`	RBF
SVM	`C`	100
SVM	`gamma`	0.01
RF	`max_depth`	2
RF	`min_samples_split`	5
RF	`n_estimators`	100

Table 2.1: Final hyper-parameter values selected for each classifier via cross-validated search.

4.2.3 Hyper-parameter Discussion

4.2.4 K-Nearest Neighbors (KNN)

- `n_neighbors` **= 1**: The model makes predictions based solely on the **single closest data point**. This creates a very complex decision boundary.
- `weights` **= distance**: Closer neighbors have a **stronger influence** on the prediction than neighbors further away. The weight of a neighbor is inversely proportional to its distance.

4.2.5 Support Vector Machine (SVM)

- `kernel` **= RBF**: Uses the **Radial Basis Function**, which allows the model to create non-linear, circular, or "blob-like" decision boundaries to separate data that isn't linearly separable.
- `C` **= 100**: A **high penalty** for misclassification. The model will prioritize classifying all training points correctly.

- `gamma` **= 0.01**: Defines the **reach** of a single training point. A low value like 0.01 means each point has a "far" reach, resulting in a smoother, more generalized decision boundary.

4.2.6 Random Forest (RF)

- `max_depth` **= 2**: Limits the trees to only **two levels deep**. This is a "shallow" forest resulting in simple logic.
- `min_samples_split` **= 5**: A node must have at least **5 data points** in it before it is allowed to split again.
- `n_estimators` **= 100**: The forest will consist of **100 individual decision trees**. Their results are averaged (for regression) or voted on (for classification) to improve overall stability.

4.3 Results and Evaluation

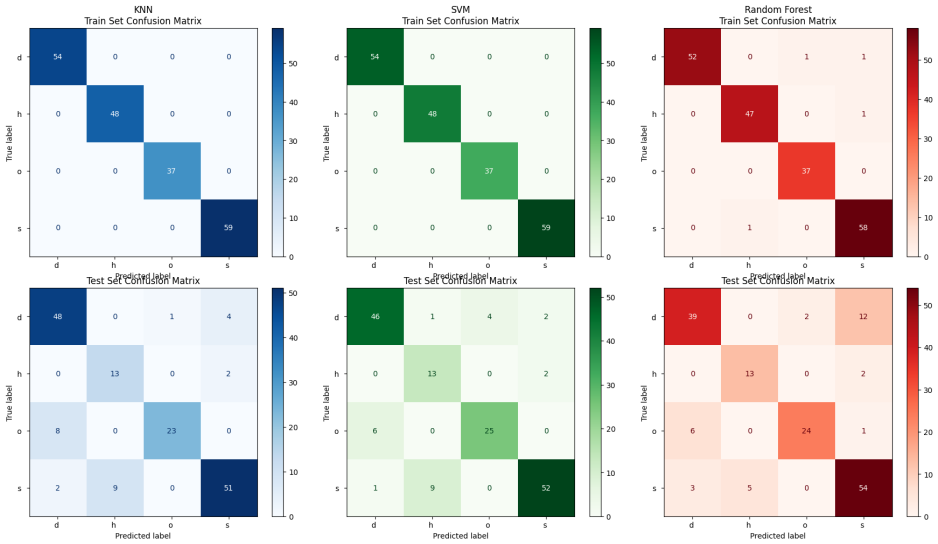


Figure 2.3 Train and test confusion matrices for KNN, SVM, and Random Forest.

Model	Train Acc.	Test Acc.	Weighted F1	Macro F1
KNN	1.000	0.839	0.84	0.82
SVM	1.000	0.845	0.85	0.82
Random Forest	0.980	0.808	0.81	0.80

Table 2.2: Classification performance for all three models on the training and test sets.

All three classifiers achieved high cross-validation scores (KNN: 97.5%, SVM: 98.0%, RF: 98.3%), yet test accuracies are consistently 15–17 percentage points lower. This gap suggests a mild distribution shift between the CV folds and the held-out test set, amplified by the small dataset size (fewer than 200 training samples).

The SVM marginally outperforms KNN (84.5% vs. 83.9%) despite both achieving 100% training accuracy, indicating both models overfit to different degrees. The OvO ensemble nature of the SVM means individual misclassifications are decided by majority vote across 6 binary classifiers, which provides some robustness over KNN's single-nearest-neighbour decision. Random Forest achieved the lowest test accuracy (80.8%), which is likely attributable to the very shallow max_depth=2 constraint limiting each tree's representational power; despite 100 estimators, the shallow ensemble is unable to capture higher-order interactions between spectral features.

Class h (Hinoki, 15 training samples) is the most challenging class across all models, with the lowest precision (KNN: 0.59, SVM: 0.57, RF: 0.72). Its small sample count means the models have less information to establish a reliable decision boundary for this class. Class o (other non-forest) achieves the highest precision across all models (KNN: 0.96, SVM: 0.86, RF: 0.92), suggesting its spectral signature is the most spectrally distinct of the four classes.

Overall, the SVM is the recommended model: it achieves the best test accuracy and benefits from its ensemble OvO structure, which provides implicit

robustness for this small, four-class problem.

5 Problem 3. Training and adapting Image Classification Deep Networks

5.1 Data Pre-processing

Images were loaded from .mat files (32×32 RGB) and normalised to [0, 1] by dividing by 255. No resizing or colour-space conversion was applied, preserving the full spatial and spectral information available at the input resolution. The training set was left in its original split (1,000 samples); class balancing for the augmented model was handled during augmentation rather than pre-processing (see Methodology).

5.2 Methodology

5.2.1 Network Design

The two models compared were a very simple model trained on non-augmented data, and a custom one of the same complexity but with varying activation and dropout trained on augmented data, inspired by in-class tutorial examples, with three paired convolutional blocks, each block consisting of two Conv2D layers followed by MaxPooling, the models are below

Simple model:

Block	Filters	Kernel	Activation	Extras
1	8	3×3	ReLU	-
2	16	3×3	ReLU	-
3	32	3×3	ReLU	-
FC	64 units	-	ReLU	-
Output	10 units	-	Softmax	-

Custom model:

Block	Filters	Kernel	Activation	Extras
1	8	3×3	Swish	BatchNorm, SpatialDropout2D(0.2)
2	16	3×3	Swish	BatchNorm, SpatialDropout2D(0.2)
3	32	3×3	Swish	BatchNorm, SpatialDropout2D(0.2)
FC	64 units	-	Swish	L1 kernel regularisation
Output	10 units	-	Softmax	-

Table 3.1: Network architecture summary. All Conv2D layers use same padding; each block ends with MaxPooling2D(2×2).

Swish activation was chosen over ReLU because it is smooth and non-monotonic, which has been shown to improve gradient flow. Batch normalisation after each convolutional block stabilises activations and accelerates convergence. SpatialDropout2D (rate=0.2) drops entire feature maps rather than individual neurons, which is more appropriate for spatially correlated convolutional features and acts as a stronger regulariser. L1 regularisation on the dense layer encourages sparse activations and further guards against overfitting on the small (1,000-sample) training set.

5.2.2 Training Approach

All models were compiled with the Adam optimiser (lr=0.0001) and SparseCategoricalCrossentropy loss, trained for up to 500 epochs with a batch size of 128, monitored against the 1,000-sample validation set. The small batch size relative to training set size (128/1000) means each epoch sees the whole training set nearly 8 times before updating the learning rate schedule. Early stopping (patience=10, restoring best weights) was used to prevent unnecessary training once validation loss stopped improving.

The low learning rate of 0.0001 was necessary because the training set is small (1,000 samples): larger learning rates caused unstable validation loss. Each epoch takes approximately 10ms, making 500-epoch training feasible without significant compute cost.

Figure 3.1 shows training and validation accuracy and loss curves for the models. The non-augmented model converges faster initially (fewer samples per epoch) but plateaus earlier as train accuracy reaches 100% and shows a larger train/val gap indicating overfitting. The custom CNN (with BatchNorm and Dropout) converges more smoothly and achieves the best validation performance with a smaller train/val gap.

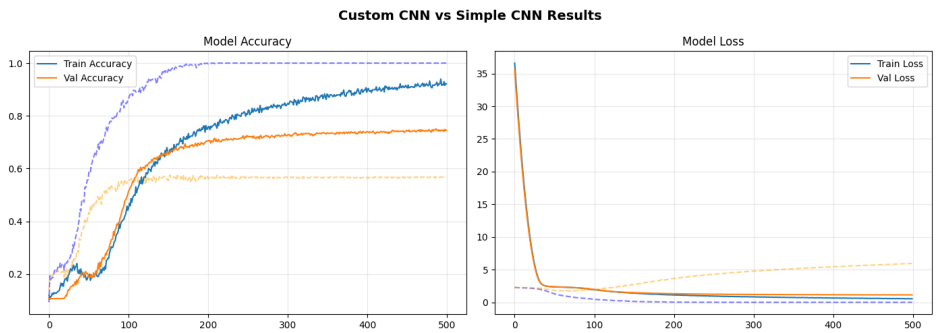


Figure 3.1 Training and validation accuracy and loss curves for models

5.2.3 Data Augmentation

To address the class imbalance in the training set (uneven distribution across 10 digit classes), a target count of 200 samples per class was set. For each under-represented class, existing samples were randomly selected and passed through an augmentation pipeline until the class reached 200 samples, expanding the total training set from 1000 to 2000 samples with balanced targets.

The augmentation pipeline consisted of:

- `RandomRotation(0.05)` - up to +/-18deg rotation
- `RandomZoom(0.025)` - up to +/-2.5% zoom
- `RandomTranslation(height=0.025, width=0.025)` - up to +/-2.5% translation in each axis

These transforms were selected because house number digits are roughly upright with minor perspective variation due to camera angle and distance; small rotations and translations reflect realistic real-world variation without introducing unrealistic distortions. Zoom was kept small to avoid cropping digit edges. Flipping was deliberately omitted because digit identities are not flip-invariant (e.g., flipping a 6 produces a 9).

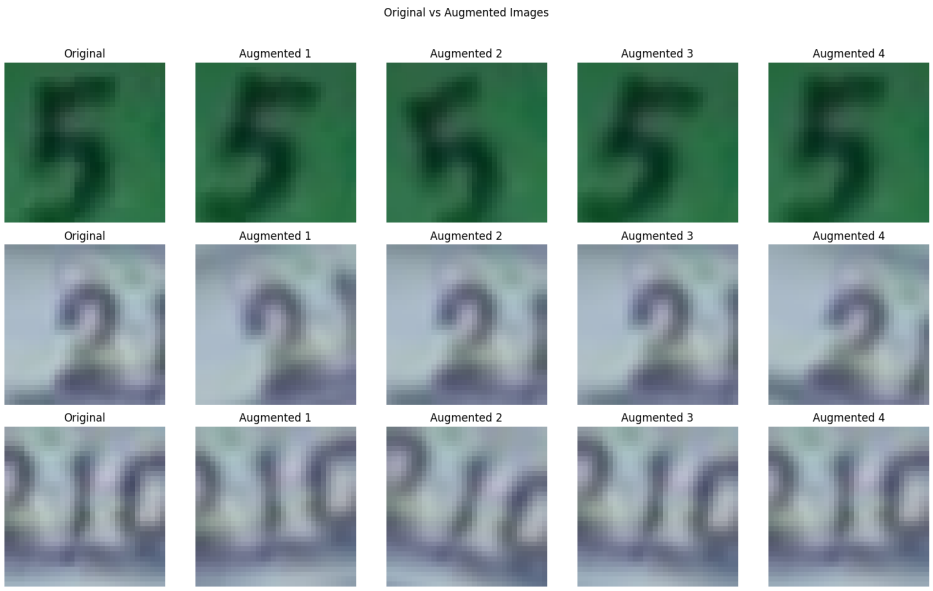


Figure 3.2 Sample images from the original and augmented training sets.

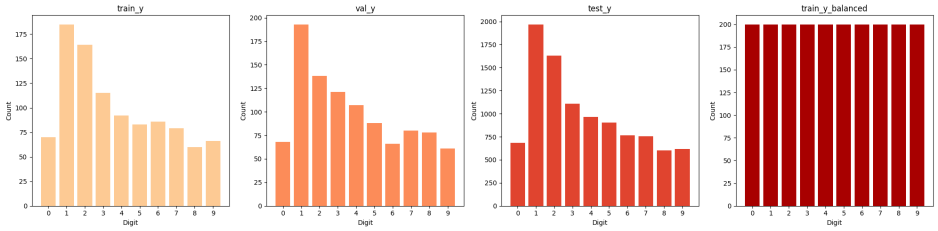


Figure 3.3 Training set class distribution before and after augmentation.

5.3 Results and Evaluation

Model	Test Accuracy	Train Time	Inference Time (test set)
SVM (baseline)	19%	2.86 s	11.30 s
Model 1: CNN, no augmentation	64%	89.69 s	0.45 s
Model 2: Custom CNN, augmentation	80%	156.73 s	0.47s

Table 3.2: Test accuracy and timing comparison across all models. SVM times measured on the vectorised flattened image representation.

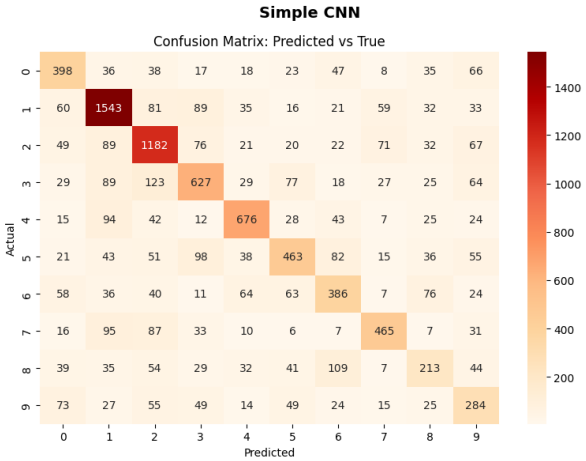


Figure 3.4 Test set confusion matrix for Simple CNN

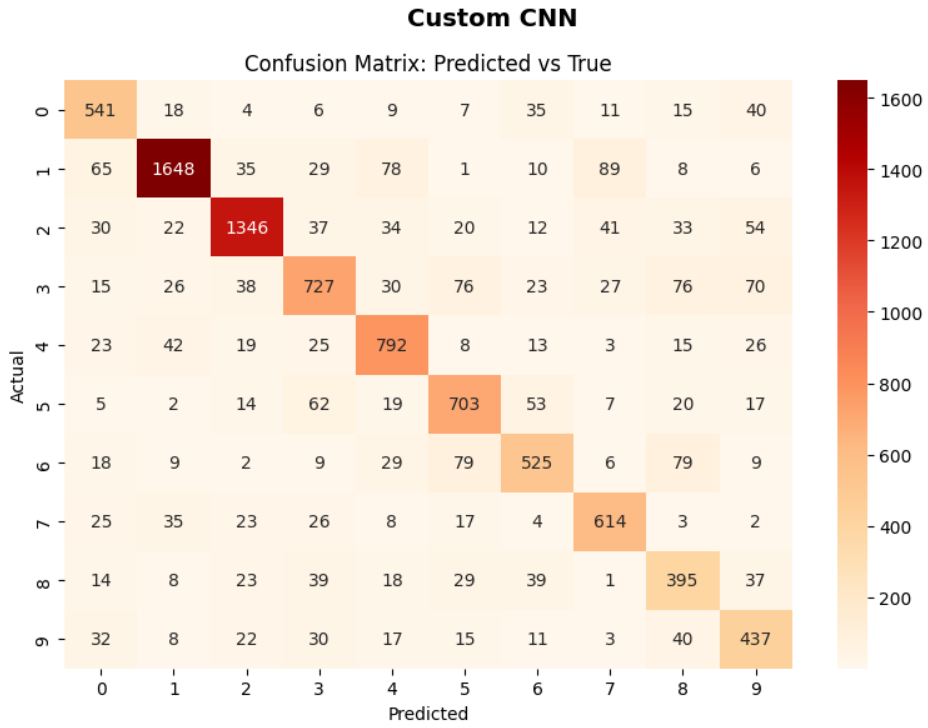


Figure 3.5 Test set confusion matrix for Custom CNN

Both CNNs outperform the SVM in test accuracy. The SVM's 11.16-second test inference time compares unfavourably to the CNN models, reflecting the cost of computing pairwise kernel evaluations for 10,000 test samples with a linear kernel trained on 3,072-dimensional vectors.

Model 2 (Custom CNN) achieves the strongest test performance. The combination of Batch Normalisation (which reduces internal covariate shift), SpatialDropout2D (which regularises correlated spatial features more effectively than standard dropout), and Swish activation (smoother gradient flow) collectively reduce overfitting and improve generalisation compared to the Simple CNN. The L1 regulariser on the dense layer further suppresses irrelevant feature combinations.

CNN inference times are significantly faster than the SVM on the full test set because CNN forward passes are implemented as batched matrix operations

on GPU/CPU BLAS kernels, whereas kernel SVM evaluation scales with the number of support vectors times the number of test samples.

6 Appendicies

6.1 Appendix A - Iterative Log-space Search for Ridge and Lasso Regression in Problem 1.

6.1.1 Ridge

```python
rmse_train_rr = []
rmse_val_rr = []
rmse_test_rr = []
coeffs_rr = []

alphas_rr = alphas.copy()
alphas_rr_history = []

for p in range(0, passes):
    start_idx = len(rmse_val_rr)

    for a in alphas_rr:
        rr_model = sm.OLS(y_train, X_train).fit_regularized(alpha=a, L1_wt=0)
        rmse_train_rr.append(np.sqrt(mean_squared_error(y_train, rr_model.predict(X_train))
))
        rmse_val_rr.append(np.sqrt(mean_squared_error(y_val, rr_model.predict(X_val))))
        rmse_test_rr.append(np.sqrt(mean_squared_error(y_test, rr_model.predict(X_test))))
        coeffs_rr.append(rr_model.params)
        alphas_rr_history.append(a)

    pass_rmse = rmse_val_rr[start_idx:]
    min_val_idx = np.argmin(pass_rmse)
    min_val_rmse = pass_rmse[min_val_idx]
    best_alpha = alphas_rr[min_val_idx]

    print(f"Pass {p + 1}/{passes} result: min RMSE: {min_val_rmse} | best alpha: {best_alph
a}")

    lower_idx = max(0, min_val_idx - 1)
    upper_idx = min(len(alphas_rr) - 1, min_val_idx + 1)

    start_val = np.log10(alphas_rr[lower_idx])
    end_val = np.log10(alphas_rr[upper_idx])

    alphas_rr = np.logspace(start_val, end_val, 10)

sorted_indices = np.argsort(alphas_rr_history)

alphas_rr_history = np.array(alphas_rr_history)[sorted_indices]
rmse_train_rr     = np.array(rmse_train_rr)[sorted_indices]
rmse_val_rr       = np.array(rmse_val_rr)[sorted_indices]
rmse_test_rr      = np.array(rmse_test_rr)[sorted_indices]
coeffs_rr         = np.array(coeffs_rr)[sorted_indices]
```

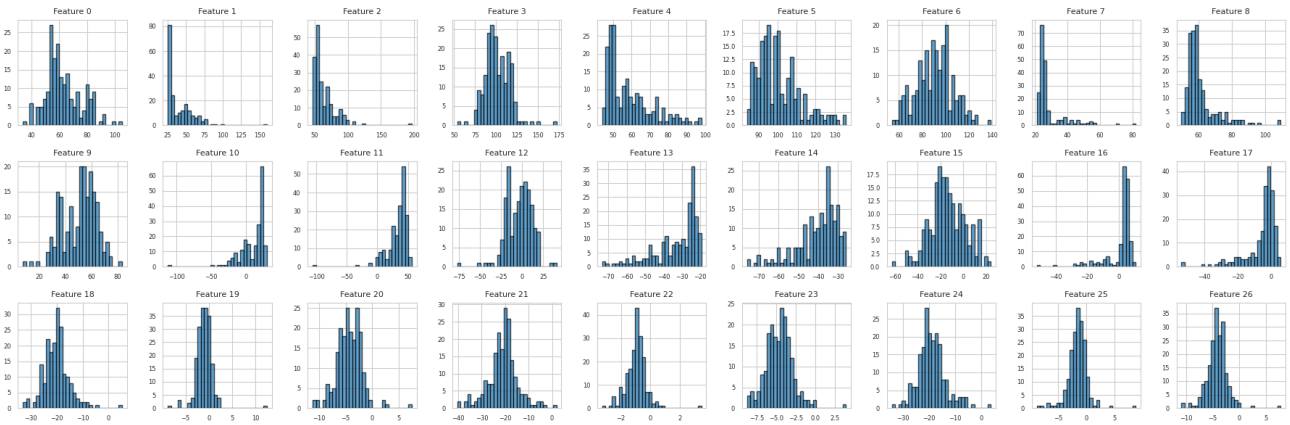
python

6.1.2 Lasso

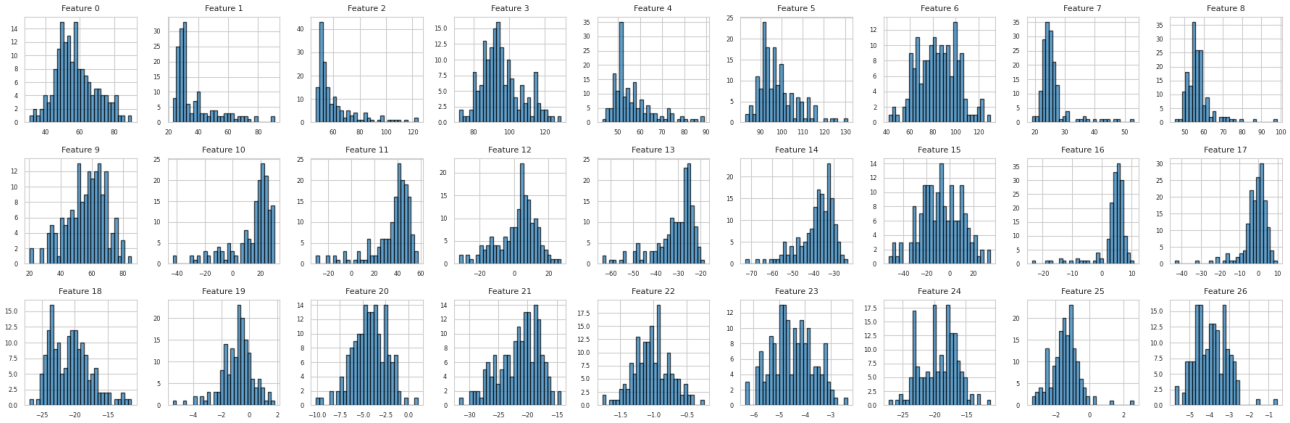
```python
rmse_train_lar = []
rmse_val_lar = []
rmse_test_lar = []
coeffs_lar = []

alphas_lar = alphas.copy()
alphas_lar_history = []

for p in range(0, passes):
    start_idx = len(rmse_val_lar)

    for a in alphas_lar:
        lar_model = sm.OLS(y_train, X_train).fit_regularized(alpha=a, L1_wt=1)
        rmse_train_lar.append(np.sqrt(mean_squared_error(y_train, lar_model.predict(X_train
))))
        rmse_val_lar.append(np.sqrt(mean_squared_error(y_val, lar_model.predict(X_val))))
        rmse_test_lar.append(np.sqrt(mean_squared_error(y_test, lar_model.predict(X_test)))
)
        coeffs_lar.append(lar_model.params)
        alphas_lar_history.append(a)

    pass_rmse = rmse_val_lar[start_idx:]
    min_val_idx = np.argmin(pass_rmse)
    min_val_rmse = pass_rmse[min_val_idx]
    best_alpha = alphas_lar[min_val_idx]

    print(f"Pass {p + 1}/{passes} result: min RMSE: {min_val_rmse} | best alpha: {best_alph
a}")

    lower_idx = max(0, min_val_idx - 1)
    upper_idx = min(len(alphas_lar) - 1, min_val_idx + 1)

    start_val = np.log10(alphas_lar[lower_idx])
    end_val = np.log10(alphas_lar[upper_idx])

    alphas_lar = np.logspace(start_val, end_val, 10)

sorted_indices = np.argsort(alphas_lar_history)

alphas_lar_history = np.array(alphas_lar_history)[sorted_indices]
rmse_train_lar     = np.array(rmse_train_lar)[sorted_indices]
rmse_val_lar       = np.array(rmse_val_lar)[sorted_indices]
rmse_test_lar      = np.array(rmse_test_lar)[sorted_indices]
coeffs_lar         = np.array(coeffs_lar)[sorted_indices]
```

python

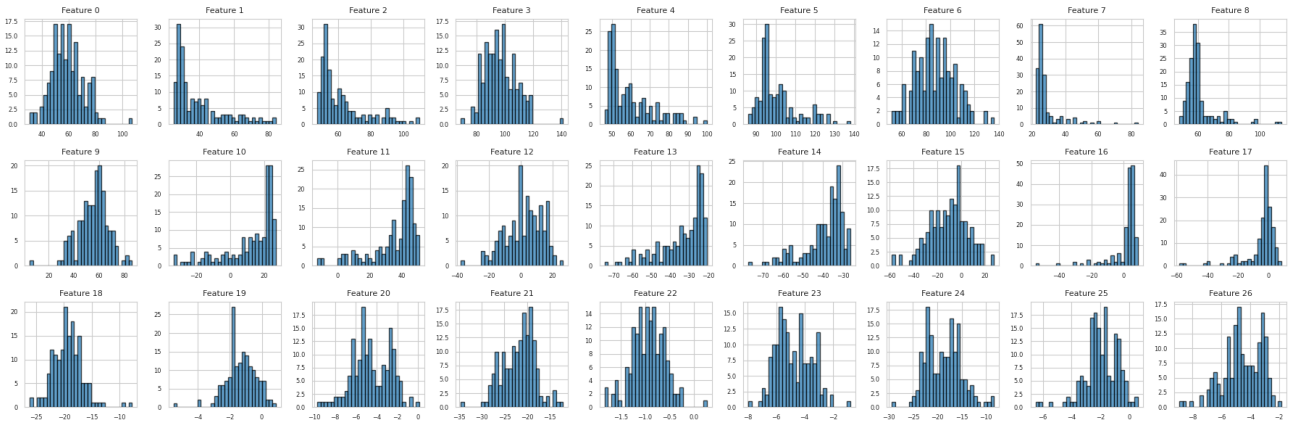
6.2 Appendix B - Histograms before and after PowerTransformer in Problem 2.



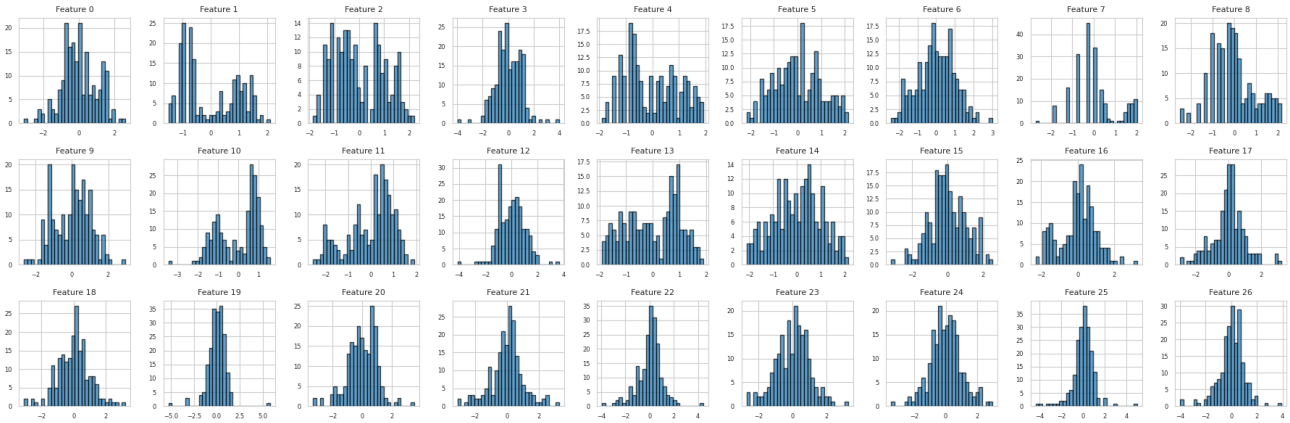
Original, train set



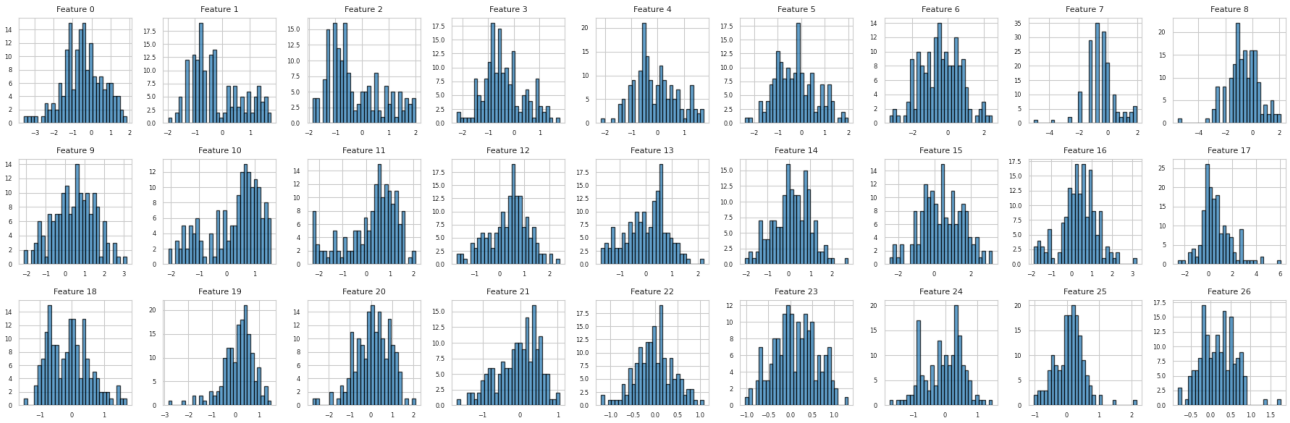
Original. val set



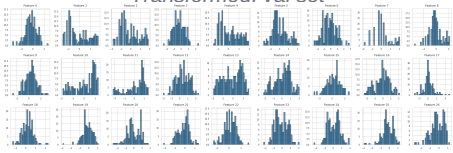
Original. test set



Transformed. train set



Transformed. val set



Transformed, test set